

PHỤ LỤC 1
HƯỚNG DẪN ĐỊNH CỠ TÀI NGUYÊN GPU

(Đi kèm guideline GL.CNVTQĐ.CNTT.18)

BẢNG THEO DÕI SỬA ĐỔI

STT	Phiên bản	Nội dung sửa đổi	Ngày hiệu lực
1	2.0	- Bổ sung các thuật ngữ, từ viết tắt liên quan - Bổ sung 3 phương pháp định cỡ: thực nghiệm, lý thuyết, tham chiếu - Bổ sung hướng dẫn quy đổi GPU - Yêu cầu hạ tầng vật lý tổng thể	30/09/2025
2	1.0	Tạo mới tài liệu	21/03/2024

I. MỤC ĐÍCH

Đưa ra hướng dẫn các bước thực hiện, phương pháp định cỡ, cấp phát tài nguyên GPU cơ bản dựa vào nhu cầu nghiên cứu, phát triển và cung cấp dịch vụ cho các hệ thống CNTT.

Lưu ý: Phụ lục được thực hiện riêng khi có nhu cầu sizing GPU, để giảm độ phức tạp của tài liệu và hồ sơ định cỡ cần đáp ứng cho các dự án.

II. KHÁI NIỆM, PHẠM VI

1. Thuật ngữ và từ viết tắt

- Định cỡ: là quá trình xác định số lượng và cấu hình cần thiết của thiết bị để đáp ứng các yêu cầu đặt ra với chi phí nhỏ nhất.

- CPU: Central Processing Unit – Bộ vi xử lý.

- GPU: Graphics Processing Unit – Bộ vi xử lý đồ họa, gia tốc tính toán song song cho AI/HPC

- VRAM: Video RAM – Bộ nhớ dành riêng cho GPU, lưu tham số, gradient, activation, optimizer states, dữ liệu đầu vào (Input Data) và thư viện/runtime.

- CFD Computational fluid dynamics: Tính toán động lực học chất lỏng, một dạng bài toán thông qua tính toán mô phỏng.

- FLOPS [Có chữ S viết hoa]: Floating Point Operations Per Second, số thao tác xử lý dấu phẩy động của CPU/GPU mỗi giây. Có các đơn vị thường dùng như TFLOPS (Tera FLOPS = 10^{12} phép tính/s), PFLOPS (Peta FLOPS = 10^{15} phép tính/s). Khả năng tính toán FLOPS phụ thuộc vào chiều dài bit của dấu phẩy động sử dụng 8, 16, 32 hay 64 bit (Floating Point: FP8, FP16, FP32 và FP 64). Khi không đề cập chiều dài bit, có thể coi số FLOPS là xử lý cho FP32 – Dấu phẩy động 32 bit.

- FLOPs [Có chữ s viết thường]: Floating Point Operations, số thao tác xử lý dấu phẩy động. Thường dùng để chỉ số phép tính cần có của hệ thống/mô hình cho mỗi lượt, mỗi lô xử lý hoặc để chỉ độ phức tạp tính toán của mô hình AI (Model Complexity).

- Iterations: Là số lượng batch cần đưa vào thực hiện để được 1 epoch.

- Bộ thông số kỹ thuật GPU: Là bộ thông số kỹ thuật đã được hãng công bố, bao gồm nhiều chỉ tiêu quan trọng liên quan đến việc sizing như VRAM, tốc độ xử lý FLOPS cho các dạng dữ liệu khác nhau FP16, FP32, FP64, INT8,... (cả cho CUDA cores và Tensor Cores với trường hợp GPU Nvidia). Dựa trên các thông số này để tính ngược số lượng GPU cần thiết

- Recipe: Trong ngữ cảnh huấn luyện LLM, “recipe” nghĩa là công thức huấn luyện – bao gồm cách chọn optimizer (AdamW, Adafactor...), batch size, gradient accumulation, learning rate schedule, precision (FP16, bfloat16, FP8...), activation checkpointing, cách song song hóa (data parallel, tensor/pipeline parallel)... Những yếu tố này quyết định hàng số compute (ví dụ 6 FLOPs mỗi

tham số mỗi token cho Transformer chuẩn) và hiệu suất tận dụng GPU. Nếu đổi recipe, hằng số đó sẽ khác.

- Time Slicing: Một kỹ thuật cho phép phân chia thời gian của GPU cho các tác vụ khác nhau, sử dụng chung tài nguyên tính toán. Lưu ý, Time Slicing không phù hợp cho huấn luyện vì quá trình huấn luyện cần tài nguyên GPU liên tục và throughput cao. Khi GPU bị chia theo lát thời gian, job huấn luyện phải liên tục ngắt – bật, gây overhead lớn do lưu/khôi phục trạng thái, làm giảm hiệu suất, kéo dài thời gian huấn luyện và khó dự đoán tiến độ.

- MIG: Multi-Instance GPU – Một kỹ thuật/giải pháp được GPU hỗ trợ, cho phép phân chia không gian 1 GPU vật lý thành nhiều “phân vùng” GPU độc lập (VRAM/SM/băng thông tách biệt) để nhiều người dùng có thể sử dụng chung 01 GPU tại cùng một thời điểm.

- Activation: Đầu ra trung gian của các lớp trong mạng; cần lưu cho giai đoạn lan truyền ngược (backpropagation)

- Activation checkpointing: Giảm VRAM bằng cách không lưu mọi activation, tính lại khi backpropagation

- Adam/AdamW/Adafactor: Các bộ tối ưu (optimizer) phổ biến trong huấn luyện

- Batch size: Số mẫu đưa vào mô hình trong mỗi bước huấn luyện (per step). Lưu ý: phân biệt per-GPU và global batch size ($= \text{per-GPU} \times \text{số GPU} \times \text{gradient accumulation}$).

- BFLOAT16 - brain floating point, một chuẩn hay dùng trong AI/ML (có phần thập phân 7 bit và phần cơ số 8 bit) khác với FLOAT16 (FP16) của IEEE (có phần thập phân 10 bit và phần cơ số 5 bit).

- FP8 / FP16 / FP32 / FP64: Các độ chính xác dấu phẩy động 8/16/32/64-bit. Độ chính xác thấp → tiết kiệm VRAM/tăng tốc nhưng cần kiểm soát ổn định.

- CUDA cores / Tensor Cores: Nhân tính toán trên GPU NVIDIA; Tensor Cores tối ưu ma trận/tensor (mixed precision).

- Data Parallelism: Sao chép mô hình trên nhiều GPU, chia batch dữ liệu; đồng bộ gradient sau mỗi step.

- Pipeline Parallelism: Chia mô hình theo chiều sâu thành nhiều stage chạy nối tiếp/chéo.

- Tensor Parallelism: Chia nhỏ phép nhân ma trận/tensor của một lớp trên nhiều GPU

- DDP (Distributed Data Parallel): Cơ chế song song theo dữ liệu của PyTorch.

- DeepSpeed: Thư viện tối ưu huấn luyện phân tán (ZeRO, pipeline, offload...).

- FSDP (Fully Sharded Data Parallel): Phân mảnh tham số/gradient/optimizer states trên nhiều GPU để tiết kiệm VRAM.

- GA (Gradient Accumulation): Cộng dồn gradient qua nhiều mini-batch để mô phỏng batch lớn khi VRAM hạn chế.
- Gradient: Đạo hàm của loss theo tham số; dùng để cập nhật trọng số.
- Learning-rate schedule: Cách thay đổi learning rate theo thời gian (cosine, step, warmup...).
- LoRA (Low-Rank Adaptation): PEFT: chỉ huấn luyện các adapter hạng thấp, giảm chi phí so với fine-tune full
- QLoRA: LoRA + nạp mô hình gốc ở dạng lượng tử hóa (thường 4-bit) để giảm VRAM.
- MLPerf: Bộ benchmark chuẩn đo hiệu năng huấn luyện/suy luận AI
- NCCL: Thư viện giao tiếp tập thể tối ưu cho GPU (all-reduce, broadcast...).
- NVLink: Liên kết tốc độ cao giữa GPU (bằng thông/độ trễ tốt hơn PCIe).
- Optimizer states: Trạng thái phụ trợ của optimizer (vd. momentum, variance). Tốn VRAM đáng kể khi huấn luyện.
- PEFT (Parameter-Efficient Fine-Tuning): Nhóm kỹ thuật fine-tune chỉ một phần nhỏ tham số (LoRA, QLoRA, adapters...).
- Perplexity: Thước đo chất lượng mô hình ngôn ngữ; càng thấp càng tốt.
- Throughput: Số samples/tokens/images xử lý mỗi giây khi huấn luyện/suy luận.
- Token / Sequence length: Đơn vị văn bản rời rạc / độ dài chuỗi đầu vào trong LLM.
- ZeRO (Zero Redundancy Optimizer): Kỹ thuật DeepSpeed loại bỏ trùng lặp lưu trữ tham số/gradient/optimizer states trên cụm.
- Compute-bound / Memory-bound / Comm-bound: Nút thắt do tính toán / băng thông/bộ nhớ / truyền thông giữa GPU; quyết định chiến lược tối ưu & cách mở rộng.

2. Phạm vi điều chỉnh

Điều chỉnh công tác định cỡ, cấp phát hạ tầng GPU trong nội bộ Tập đoàn và các hệ thống phục vụ kinh doanh, triển khai cho khách hàng ngoài, Guideline áp dụng để định cỡ cho hệ thống CNTT có sử dụng GPU.

Các tài nguyên định cỡ bao gồm: GPU, VRAM.

Guideline không xác định cho các bài toán CFD, mô phỏng và tạo ảnh (rendering) có sử dụng GPU do các bài toán này được tính toán riêng và có thể theo khuyến nghị từ hãng phần mềm mô phỏng/render.

3. Đối tượng áp dụng

Guideline áp dụng xuyên suốt đối với Khối cơ quan Tập đoàn, các Tổng Công ty, Công ty, Trung tâm, Viện, Học viện Viettel và các đơn vị hạch toán phụ thuộc Tập đoàn trong công tác định cỡ, cấp phát hạ tầng cho các dự án và hệ thống CNTT.

III. TÀI LIỆU THAM CHIẾU

Các tài liệu có tham chiếu và được sử dụng trong guideline gồm:

- Guideline phân chia mức độ quan trọng hệ thống ứng dụng công nghệ thông tin: GL.CNVTQĐ.CNTT.03

IV. NỘI DUNG

1. Nguyên tắc định cỡ

Xác định rõ nhu cầu cho mục đích của ứng dụng:

- Định cỡ cho hệ thống cung cấp dịch vụ (Production), cần xác định nhu cầu dự phòng, theo các quy định hiện hành về dự phòng.
- Định cỡ cho hệ thống Lab phục vụ nghiên cứu phát triển, môi trường phát triển, test, do chi phí lớn nên không bắt buộc xác định nhu cầu dự phòng.

Xác định tài nguyên sát với thực tiễn nhất có thể dựa trên:

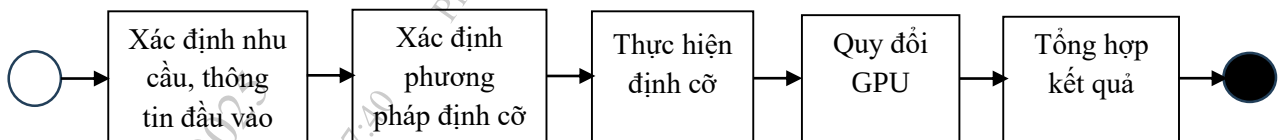
- Nguồn thông tin tài nguyên từ các bài toán, hệ thống tương tự
- Thông tin với các bài toán, mô hình AI/ML đã được kiểm chứng và công bố thông tin.

Xác định qua giai đoạn bài toán AI/ML:

- Training – Huấn luyện: Hiệu chỉnh, tối ưu mô hình (model) theo dữ liệu được đào tạo, hoặc tự đào tạo.
- Inference – Suy luận: Đánh giá khả năng của mô hình, khai thác kết quả mô hình.

Để đơn giản, xác định tổng tài nguyên theo từng nhu cầu tài nguyên thành phần tại cùng một thời điểm bằng cộng cơ học. Ví dụ một hệ thống cần 02 bước thành phần thực hiện giải đoán cùng lúc, tài nguyên sẽ bằng tổng nhu cầu của cả 2 thành phần.

Tóm tắt các bước tính toán:



2. Định cỡ cho Huấn luyện (Training)

a) Nhu cầu và thông tin đầu vào.

- Mô hình: Lựa chọn kiến trúc và kích thước (ví dụ: Llama 3.2 - 70B tham số, ResNet-50).
- Quy mô dữ liệu & Dạng dữ liệu: Số lượng bản ghi/msg, kích thước dữ liệu, số lượng tham số; Video, Ảnh, Text, âm thanh... (ví dụ: 1 tỷ token tiếng Việt, 10.000 ảnh độ phân giải 14.896 x 14.896 pixel)
- Thời gian huấn luyện tối đa (ví dụ: 30 ngày)
- Phương pháp huấn luyện:
 - o Huấn luyện toàn bộ (Full Training).

- Fine-tuning.
- Parameter-Efficient Fine-Tuning: LoRA hoặc QLoRA để giảm bộ nhớ và chi phí tính toán.

- Chọn loại GPU cần xác định số lượng

b) Xác định phương pháp định cỡ cho Huấn luyện.

Ba phương pháp chính được áp dụng để định cỡ cho huấn luyện bao gồm:

Thực nghiệm: Đo đạc trực tiếp trên hệ thống thực hoặc môi trường thử nghiệm bằng workload đại diện. Thu thập dữ liệu về thông lượng (throughput), độ trễ, mức sử dụng bộ nhớ GPU và tài nguyên hệ thống. Đây là phương pháp sát thực tế nhất, được các nhà cung cấp phần cứng (như NVIDIA¹) khuyến nghị sử dụng để ra quyết định cuối cùng.

Lý thuyết: Sử dụng mô hình toán học dựa trên đặc tính phần cứng, phần mềm và thuật toán để ước lượng. Phương pháp này hữu ích ở giai đoạn tiền đầu tư khi chưa có hệ thống để thử nghiệm

Tham chiếu: Dựa trên so sánh đối chiếu với bài toán tương tự (các bộ chuẩn công khai, kết quả thử nghiệm từ cộng đồng hoặc kết quả chạy mô hình AI tương tự đã có tại Viettel).

Ưu tiên phương pháp thực nghiệm để đo kết quả chính xác nhất.

Trong trường hợp chưa có môi trường thực nghiệm, sử dụng phương pháp tham chiếu hoặc lý thuyết để ước lượng sơ bộ.

c) Định cỡ theo phương pháp Thực nghiệm

Bước 1: Xác định phạm vi thực nghiệm

- Tập con dữ liệu: Chọn một tỷ lệ nhỏ của toàn bộ dữ liệu để giảm thời gian mỗi lần thử nghiệm. Ví dụ: 5% của dữ liệu gốc. Lưu ý, tỷ lệ càng cao thì kết quả đo càng chính xác.

- Thời gian chạy mỗi thực nghiệm: Xác định thời lượng cần thiết để quá trình thực nghiệm chạy ổn định (vì thời gian đầu sẽ là thời gian khởi tạo, nạp dữ liệu... sẽ gây nhiễu).

- Mục tiêu đo lường: Mức sử dụng bộ nhớ GPU, thời gian huấn luyện.

- Loại GPU dùng để thử nghiệm

Bước 2: Thiết lập môi trường thực nghiệm

- Cố định random seed.

- Cố định batch size, optimizer, learning rate.

- Cố định kích thước dữ liệu đầu vào (độ dài chuỗi, kích thước ảnh...)

- Cố định các bước xử lý dữ liệu.

- Cố định phiên bản framework (Pytorch, TensorFlow, DeepSpeed...) và phiên bản dữ liệu.

Bước 3: Thực nghiệm

¹ <https://docs.nvidia.com/vgpu/sizing/virtual-workstation/latest/methodology.html>

- Với loại GPU đã chọn để thử nghiệm, huấn luyện thử với số GPU tăng dần:

$$N_{GPU} = 1, 2, 3 \dots N_{GPU-Max}$$

- Trong đó $N_{GPU-Max}$ là số GPU tối đa có thể thực nghiệm tại môi trường thực nghiệm.
- Trường hợp VRAM của 1 GPU không đủ để huấn luyện thì phải dùng Model Parallelism (kỹ thuật được sử dụng khi một mô hình AI quá lớn để chứa vừa trong VRAM của một GPU. Mô hình sẽ được "cắt" ra và mỗi mảnh được đặt trên một GPU khác nhau. Trong trường hợp này, tổng dung lượng VRAM của toàn bộ cụm GPU phải lớn hơn kích thước mô hình), bắt đầu với số lượng n GPU tối thiểu đáp ứng VRAM để huấn luyện và tăng dần GPU từng lần thử nghiệm theo bội số của n ($n, 2n, 3n, \dots$)

- Khởi động (warm up): Chạy quá trình huấn luyện trong một số lượng vòng lặp (step/iteration) cố định mà không đo lường. Mục đích để hệ thống (GPU, phần mềm, luồng dữ liệu) đi vào trạng thái hoạt động ổn định. Ví dụ: Cho mô hình chạy 200 steps đầu tiên và bỏ qua các kết quả. Lưu ý: bỏ toàn bộ số liệu trong pha warm-up.

- Lặp lại 2 – 3 lần cho mỗi cấu hình (việc chỉ 1 lần sẽ có thể nhiễu do biến thiên do hệ thống, cần lặp lại để lấy trung bình + sai số.)

- Với mỗi lần thực nghiệm, thực hiện thu thập: Throughput (samples/s, tokens/s, images/s). Thống kê thông lượng trung bình các lần chạy của mỗi cấu hình.

Ví dụ:

N_GPU	Throughput (images/s)
1	12
2	21
3	30
4	38
5	47
6	54
7	61
8	68

N_GPU	Throughput (images/s)
9	74
10	80
11	86
12	92
13	98
14	103
15	108
16	114

Bước 4: Suy diễn

- Từ dữ liệu thực nghiệm, dựng hàm/mô hình mở rộng throughput (ví dụ: Amdahl's law²).

- Tìm N_{GPU} thỏa mãn:

² Amdahl's Law | SpringerLink, <https://www.kth.se/blogs/pdc/2018/11/scalability-strong-and-weak-scaling/>

$$T(N_{GPU}) = \frac{N_{samples}}{Throughput(N_{GPU})} \leq T_{target}$$

Trong đó:

- $T(N_{GPU})$: thời gian huấn luyện ước lượng với N_{GPU} ;
- T_{target} là thời gian huấn luyện mục tiêu.

Ví dụ: Từ dữ liệu thực nghiệm thu thập được, cần xây dựng một mô hình mô tả mối quan hệ giữa số lượng GPU và thông lượng (throughput). Quan sát bảng số liệu cho thấy khi số GPU còn ít, throughput tăng gần tuyến tính, tuy nhiên khi số GPU tăng cao, tốc độ gia tăng throughput giảm dần. Nguyên nhân chủ yếu đến từ chi phí đồng bộ, truyền thông giữa các GPU và phân tác vụ không thể song song hóa được. Một mô hình phù hợp và có thể giải thích rõ ràng hiện tượng này là Amdahl's Law, vốn được dùng để mô hình hóa hiệu năng song song trong các hệ thống tính toán. Theo Amdahl's Law, throughput có thể được biểu diễn như sau:

$$Throughput(N_{GPU}) = \frac{Throughput(1)}{s + \frac{1-s}{N_{GPU}}}$$

Trong đó s là tham số cần tìm, đại diện cho tỷ lệ tác vụ tuần tự không song song hóa được (tương tự $1-s$ là đại diện cho tỷ lệ tác vụ song song).

Để sử dụng mô hình này cho bài toán ước lượng số GPU cần thiết, cần ước lượng tham số s cho mô hình Amdahl's Law, sử dụng phương pháp Nonlinear Least Squares (NLS) để khớp đường cong Amdahl với dữ liệu throughput thực nghiệm. NLS hoạt động dựa trên nguyên lý tối thiểu hóa sai số bình phương giữa giá trị đo thực nghiệm và giá trị dự đoán của mô hình, lặp lại cho đến khi hội tụ. Kết quả thu được như sau:

$$Throughput(N_{GPU}) = \frac{12}{0.055 + \frac{1-0.055}{N_{GPU}}}$$

Ta cần tìm N_{GPU} sao cho thỏa mãn:

$$\frac{N_{samples}}{Throughput(N_{GPU})} \leq T_{target}$$

Giả sử yêu cầu đầu vào là $N_{samples} = 1000000$ ảnh; $T_{target} = 2h \sim 7200s$, ta được:

$$N_{GPU} = \left\lceil \frac{1-s}{Throughput(1) * \frac{T_{target}}{N_{samples}} - s} \right\rceil = [30.09] = 31 GPU$$

$$\text{với điều kiện } \frac{N_{samples}}{T_{target}} - \frac{Throughput(1)}{s} > 0$$

Lưu ý: Trong quá trình mở rộng số lượng GPU, cần cân nhắc kỹ lưỡng giữa mục tiêu rút ngắn thời gian huấn luyện và chi phí phát sinh. Hiệu quả mở rộng thường giảm dần theo số GPU: mỗi GPU bổ sung chỉ mang lại mức cải thiện thông lượng nhỏ hơn so với GPU trước đó. Khi vượt qua ngưỡng hiệu quả (knee point), chi phí tăng lên nhanh chóng trong khi lợi ích thu được không đáng kể. Vì vậy, việc xác định điểm dừng hợp lý là cần thiết nhằm tối ưu đồng thời hiệu năng huấn luyện và chi phí vận hành.

d) Định cỡ theo phương pháp Lý thuyết

Bước 1: Tính số lượng GPU dựa trên nhu cầu tính toán³:

Thời gian huấn luyện ước lượng được mô tả như sau:

$$T(N_{GPU}) = \frac{FLOPS_{total}}{FLOPS_{single-GPU} * N_{effective_GPU}}$$

Trong đó:

- $T(N_{GPU})$: Thời gian huấn luyện ước lượng khi dùng N_{GPU} GPU
- $FLOPS_{total}$: Tổng số phép toán dấu phẩy động cần để huấn luyện mô hình.
- $FLOPS_{single-GPU}$: Năng lực tính toán lý thuyết (số FLOPs/s) của một GPU do nhà cung cấp công bố tương ứng với độ chính xác (Precision) đang được sử dụng trong huấn luyện (FP32, FP16, BFLOAT16, Tensor Core, hay CUDA Core). Lấy số liệu này ứng với loại GPU đang cần xác định số lượng.
- $N_{effective_GPU}$: Số GPU hiệu dụng, tính theo hiệu quả mở rộng khi huấn luyện dùng nhiều GPU, thông thường được mô hình hóa bằng công thức sau:

$$N_{effective_GPU} = N_{GPU} * 0.85$$

0.85 là giá trị thực nghiệm, quan sát được từ các thử nghiệm với hệ thống phân tán được tối ưu (như PyTorch DDP/FSDP, DeepSpeed) trên hạ tầng có băng thông cao (ví dụ NVLink). Lưu ý con số 85% không cố định, có thể cao hơn với tác vụ nặng về tính toán, interconnect tốt, thư viện tối ưu (như NCCL); hoặc thấp hơn nhiều nếu tác vụ phụ thuộc nhiều vào giao tiếp, interconnect chậm, hoặc cài đặt kém tối ưu.

Để tìm số lượng GPU thỏa mãn yêu cầu thời gian huấn luyện tối đa là T_{target} , thực hiện giải bất phương trình $T(N_{GPU}) \leq T_{target}$, ta có kết quả (công thức đã hiệu chỉnh một số hằng số) như sau:

$$N_{GPU} = \left\lceil \frac{FLOPS_{total}}{T_{target} * FLOPS_{single-GPU} * 0.85} \right\rceil$$

³ <https://apxml.com/posts/how-to-calculate-vram-requirements-for-an-llm>

Công thức trên phụ thuộc chủ yếu ở $FLOPs_{total}$ vốn được xác định bởi nhiều yếu tố như dữ liệu đầu vào, kiến trúc mô hình.

- Với LLM dạng Transformer decoder-only, tổng FLOPs có thể ước lượng bằng công thức như sau⁴:

$$FLOPs_{total} = 8 * N_{params} * N_{tokens}$$

Trong đó N_{params} , N_{tokens} lần lượt là số lượng tham số mô hình và tổng số token dùng để huấn luyện.

- Với các loại mô hình cho các bài toán khác như CV, RecSys... cách tính FLOPs sẽ khác (ví dụ: phụ thuộc vào độ phân giải ảnh, embedding table...). Trong trường hợp vẫn muốn dùng phương pháp lý thuyết, cần nghiên cứu tài liệu chuyên ngành hoặc tự giải trình chi tiết.

Bước 2: Tính số lượng GPU dựa trên nhu cầu về bộ nhớ⁵

Khi sizing cần tính toán VRAM vì đây là tài nguyên giới hạn quyết định mô hình và dữ liệu có thể nạp vừa vào VRAM của GPU hay không. VRAM cũng ảnh hưởng trực tiếp đến batch size, throughput, latency, từ đó chi phối cả hiệu năng và chi phí phần cứng.

Tính toán tổng bộ nhớ VRAM cần thiết để huấn luyện.

- Đối với training hoặc fine-tuning mạng neuron, thực hiện tính VRAM theo phương pháp mô tả dưới đây
- Đối với các thuật toán khác (như XGBoost, LightGBM...) cần nghiên cứu tài liệu chuyên ngành hoặc tự giải trình chi tiết

Tính toán VRAM đối với training hoặc fine-tuning mạng neuron:

$$VRAM = VRAM_{params} + VRAM_{optimizer} + VRAM_{gradient} + VRAM_{activation} + VRAM_{multi-GPU\ overhead}$$

Trong đó

- VRAM để lưu tham số mô hình (Model Parameters):

$$VRAM_{params} = N_{params} * Bpp$$

- N_{params} : Tổng số tham số của mô hình. Ví dụ: 7B, 70B, 180B
- Bpp : Số byte mỗi tham số:
 - FP32: 4 byte
 - FP16/BF16: 2 byte
 - INT8: 1 byte
 - INT4: 0.5 byte

Ví dụ: Mô hình 7B tham số ở FP16:

$$VRAM_{params} = 7 * 10^9 * 2 = 14\ GB$$

⁴ <https://developer.nvidia.com/blog/scaling-language-model-training-to-a-trillion-parameters-using-megatron/>

⁵ <https://apxml.com/posts/how-to-calculate-vram-requirements-for-an-llm>

- VRAM để lưu Trạng thái bộ tối ưu (Optimizer States): Khi huấn luyện, các bộ tối ưu (optimizer) lưu trữ trạng thái cho mỗi tham số có thể huấn luyện. Cụ thể công thức như sau:

$$VRAM_{optimizer} = N_{state} * N_{params} * Bpp$$

- N_{state} : Số trạng thái của optimizer.

Ví dụ: Fine-tuning 7B tham số với Adam (lưu 2 trạng thái momentum và variance ở FP32):

$$VRAM_{params} = 2 * 7 * 10^9 * 4 \text{ (bytes)} = 56 \text{ GB}$$

Lưu ý: Có thể giảm bằng các optimizer 8-bit (DeepSpeed, bitsandbytes)

- VRAM để lưu Gradient: Khi huấn luyện, gradients được tính cho mỗi tham số khi lan truyền ngược (backpropagation), thường có cùng độ chính xác với tham số.

$$VRAM_{gradient} = N_{params} * Bpp$$

Ví dụ: Mô hình 7B tham số ở FP16:

$$VRAM_{params} = 7 * 10^9 * 2 \text{ (bytes)} = 14 \text{ GB}$$

- VRAM để lưu Activations: Activations là các đầu ra trung gian của các lớp khi lan truyền xuôi, kích thước phụ thuộc vào nhiều yếu tố như batch size, kích thước đầu vào mô hình, kiến trúc mô hình cũng như các kỹ thuật tối ưu hóa như activation checkpointing nên rất khó xác định. Công thức dưới đây chỉ là một ước lượng thô đặc thù cho các mô hình Transformer và được dùng như một hướng dẫn tham khảo, đối với các mô hình khác không áp dụng được. Trong trường hợp vẫn muốn dùng phương pháp lý thuyết, cần nghiên cứu tài liệu chuyên ngành hoặc tự giải trình chi tiết:

$$VRAM_{activation} = K * N_{batch_size} * N_{seq_len} * N_{hidden_dim} * N_{layer} * Bpp$$

Trong đó:

- N_{batch_size} : Số lượng chuỗi được xử lý cùng lúc
- N_{seq_len} : Độ dài chuỗi đầu vào
- N_{hidden_dim} : Số chiều tầng ẩn
- N_{layer} : độ sâu của mô hình
- K là hệ số ước lượng phụ thuộc vào từng mô hình (thông thường giá trị trong khoảng từ 10 đến 30)

Ví dụ: Mô hình Llama 7B⁶ có $N_{hidden_dim} = 4096, N_{layer} = 32$; huấn luyện với độ chính xác FP8; $N_{seq_len} = 2048$ (giá trị phổ biến); $N_{batch_size} = 16$

$$VRAM_{activation} = 30 * 16 * 2048 * 4096 * 32 * 1 = 120 \text{ GB}$$

- VRAM phụ trội khi sử dụng nhiều GPU: Việc sử dụng nhiều GPU sẽ phát sinh chi phí dư thừa (overhead) so với chỉ dùng một GPU, nghĩa là hiệu năng và

⁶ <https://arxiv.org/html/2312.04333v4>

bộ nhớ không thể mở rộng tuyến tính hoàn hảo. Nguyên nhân chủ yếu đến từ nhu cầu giao tiếp và đồng bộ giữa các GPU. Thông thường, bộ nhớ bổ sung khi sử dụng nhiều GPU có thể ước lượng như sau:

$$VRAM_{multi-GPU\ overhead} = VRAM_{single-GPU} * 0.05 * \sqrt{N_{GPU} - 1}$$

Lưu ý: Hệ số 0.05 và cách mô hình hóa theo căn bậc hai là giả định đơn giản để ước lượng nhanh, thực tế overhead bộ nhớ phụ thuộc rất nhiều vào chiến lược song song (Data, Tensor, Pipeline Parallel và ZeRO stage), hạ tầng kết nối và framework.

Ví dụ: Huấn luyện trên 16 GPU B200 (192GB):

$$VRAM_{multi-GPU\ overhead} = 192 * 0.05 * \sqrt{16 - 1} = 37\ GB$$

Sau khi tính số lượng VRAM cần có, có thể dựa trên dung lượng VRAM của card GPU có sẵn trên thị trường để xác định số lượng card GPU hoặc server cần thiết.

$$N = \frac{VRAM_{total}}{0.7 * VRAM_{gpu}}$$

Trong đó $VRAM_{total}$ là tổng bộ nhớ cần có, $VRAM_{gpu}$ là VRAM của loại GPU cần xác định số lượng, 0.7 là hệ số hiệu dụng bộ nhớ cho VRAM mỗi GPU (Thực tế luôn có phần không shard được hoặc shard không đều).

Bước 3: Tính số lượng GPU cần đáp ứng bài toán

Số lượng GPU cần dùng phải đáp ứng cả nhu cầu tính toán (bước 1) và nhu cầu về bộ nhớ (bước 2).

Số GPU cần dùng = Max (Số lượng GPU xác định tại bước 1, Số lượng GPU xác định tại bước 2)

e) Tham chiếu

Dữ liệu tham chiếu có thể lấy từ các nguồn sau:

1. Công bố từ các nguồn uy tín trên thế giới (bài báo, trang web, công bố của hãng,...), vd: <https://www.nvidia.com/en-us/data-center/resources/mlperf-benchmarks>

2. Kết quả từ thực nghiệm

3. Kết quả từ việc thực hiện mô hình tương đương tại Viettel

Chỉ áp dụng phương pháp này khi có bài toán tham chiếu và bài toán cần định cỡ sử dụng mô hình dựa trên mạng neuron và chung một loại mô hình (ví dụ Llama, Deepseek), chỉ khác nhau về các điều kiện:

- Kích Thước mô hình (Model Size): Số lượng tham số (Parameters)
- Kích thước dữ liệu huấn luyện: Dataset size

Giả sử Baseline bài toán A đã biết là: mô hình A (có số lượng tham số là $Params_A$), cần dùng số lượng N_A GPU loại g, huấn luyện trên tập dữ liệu có D_A mẫu dữ liệu, Trong thời gian T_A .

Yêu Cầu (Bài toán B): mô hình B (có số lượng tham số là $Params_B$), huấn luyện trên tập dữ liệu có D_B mẫu dữ liệu, Trong thời gian T_B . Nếu cùng sử dụng loại GPU g giống như bài toán A thì cần số lượng là bao nhiêu?

Số GPU loại g cần thiết để giải bài toán B

$$N_B = N_A * \frac{PARAM_B}{PARAM_A} * \frac{D_B}{D_A} * \frac{T_A}{T_B}$$

Công thức này giả định rằng recipe không đổi và sự khác biệt về số Params hoặc Dataset không quá lớn.

- Trường hợp recipe thay đổi thì điều chỉnh công thức quy đổi dựa trên đặc tính thay đổi tương ứng trong recipe kèm theo giải trình chi tiết.
- Chú ý về tính chính xác nếu có sự khác biệt lớn về Params hoặc Dataset.

Lưu ý: Trong trường hợp mô hình không dựa trên mạng neuron, cần nghiên cứu tài liệu chuyên ngành hoặc tự giải trình chi tiết cách thức so sánh tương quan phù hợp.

3. Định cỡ cho Suy luận (Inference)

a) Nhu cầu và thông tin đầu vào

- Mô hình: Lựa chọn kiến trúc và kích thước (ví dụ: Llama 3.2 - 70B tham số, ResNet-50)

- Dữ liệu thực nghiệm cho Suy luận: Xác định đặc tính và quy mô dữ liệu đầu vào để thử nghiệm (ví dụ: 1000 - 3000 câu tiếng Việt độ dài trung bình 128 token; 300-1.000 ảnh độ phân giải 1.024 x 1.024 pixel; 200 đoạn video dài 30 giây ở độ phân giải 1080p).

- Xác định nhu cầu của bài toán, thời gian, độ trễ yêu cầu cho mỗi lượt xử lý, các thông số kỹ thuật, KPI liên quan..., ảnh hưởng đến định cỡ

- Các thông tin triển khai ảnh hưởng đến việc sizing như: triển khai đơn (GPU dành riêng) hay dùng chung (Time Slicing, MIG...)

b) Xác định phương pháp Suy luận

Hai phương pháp chính được áp dụng để định cỡ cho Suy luận bao gồm:

Thực nghiệm: Đo đạc trực tiếp trên hệ thống thực hoặc môi trường thử nghiệm bằng workload đại diện. Thu thập dữ liệu về thông lượng (throughput), độ trễ, mức sử dụng bộ nhớ GPU và tài nguyên hệ thống. Đây là phương pháp sát thực tế nhất, được các nhà cung cấp phần cứng (như NVIDIA⁷) khuyến nghị sử dụng để ra quyết định cuối cùng.

Tham chiếu: Dựa trên so sánh đối chiếu với bài toán tương tự (các bộ chuẩn công khai, kết quả thử nghiệm từ cộng đồng hoặc kết quả chạy mô hình AI tương tự đã có tại Viettel).

Ưu tiên phương pháp thực nghiệm để đo kết quả chính xác nhất.

⁷ <https://docs.nvidia.com/vgpu/sizing/virtual-workstation/latest/methodology.html>

Trong trường hợp chưa có môi trường thực nghiệm, sử dụng phương pháp tham chiếu để ước lượng sơ bộ.

c) Phương pháp thực nghiệm

Bước 1: Xác định phạm vi thực nghiệm

- Tập con dữ liệu: Chọn một tập nhỏ đại diện cho workload suy luận thực tế (ví dụ: 1.000–3.000 câu tiếng Việt độ dài trung bình 128 token; 300–1.000 ảnh 1.024×1.024 pixel). Tập càng lớn thì kết quả đo càng chính xác.
- Thời gian chạy mỗi thực nghiệm: Quy định khoảng thời gian đủ dài để hệ thống ổn định (tránh nhiễu do giai đoạn khởi tạo, nạp mô hình hoặc warm-up).
- Mục tiêu đo lường: Độ trễ (p50/p95), thông lượng (request/giây), mức sử dụng bộ nhớ/tài nguyên GPU–CPU, tỷ lệ lỗi.
- Loại GPU dùng để thử nghiệm

Bước 2: Thiết lập môi trường thực nghiệm

- Cố định random seed để đảm bảo kết quả đo có thể tái hiện (đặc biệt với mô hình sinh chuỗi).
- Cố định batch size hoặc chính sách batching (static/dynamic) để so sánh công bằng giữa các lần chạy.
- Cố định độ dài dữ liệu đầu vào/đầu ra (số token, kích thước ảnh, độ dài video) nhằm loại bỏ biến động không kiểm soát.
- Cố định pipeline xử lý dữ liệu (tiền xử lý, chuẩn hóa, nạp dữ liệu).
- Cố định phiên bản framework và runtime (PyTorch, TensorRT, vLLM, ONNX Runtime, Triton...) cũng như phiên bản mô hình và dữ liệu thử nghiệm.

Bước 3: Thử nghiệm

- Chạy tải tăng dần trên 01 GPU.
 - o Mục tiêu: tìm ra số request/giây tối đa mà 1 GPU có thể xử lý trong khi vẫn giữ độ trễ theo yêu cầu
 - o Thực hiện:
 - Lấy 1 GPU (ví dụ A100).
 - Khởi động (warm up): Chạy quá trình suy luận trong một số lượng request cố định mà không đo lường. Mục đích để hệ thống (GPU, phần mềm, luồng dữ liệu) đi vào trạng thái hoạt động ổn định. Ví dụ: Cho mô hình chạy 200 request đầu tiên và bỏ qua các kết quả.
 - Bắt đầu gửi request vào với tốc độ $n_{\text{test}} = 1, 2, 4, 8 \dots$ request/giây.
 - Tại mỗi mức n_{test} , đo t_{latency} thực tế.
 - Tiếp tục tăng n_{test} cho đến khi t_{latency} đo được chạm ngưỡng KPI cho phép.
 - Lặp lại 2 – 3 lần cho mỗi cấu hình (chỉ 1 lần sẽ có thể nhiễu do biến thiên do hệ thống, cần lặp lại để lấy trung bình + sai số). Với mỗi lần thu thập: Throughput (samples/s, tokens/s, images/s). Xác định thông lượng trung bình của mỗi cấu hình.

- Kết quả: xác định được thông lượng max khi sử dụng 01 GPU, Ví dụ: xác định được 1 GPU A100 có thể xử lý tối đa 20 request/giây mà vẫn giữ được độ trễ dưới mức yêu cầu.
- Tăng dần số GPU 1, 2, 3, 4,..., với mỗi lần thực nghiệm, thực hiện thu thập:

- Độ trễ, tỉ lệ lỗi: đảm bảo yêu cầu đầu vào
- Throughput (samples/s, tokens/s, images/s)

Ví dụ:

N_GPU	Throughput (request/s)
1	12
2	21
3	30
4	38
5	47
6	54
7	61
8	68

N_GPU	Throughput (request/s)
9	74
10	80
11	86
12	92
13	98
14	103
15	108
16	114

Bước 4: Suy diễn

Thực hiện tương tự theo phương pháp suy diễn trong bước huấn luyện tại mục IV.2.c để tìm ra số GPU cần thiết đáp ứng được yêu cầu đầu vào.

d) Tham chiếu

Dữ liệu tham chiếu có thể lấy từ các nguồn sau:

1. Công bố từ các nguồn uy tín trên thế giới (bài báo, trang web, công bố của hãng,...), vd: <https://www.nvidia.com/en-us/data-center/resources/mlperf-benchmarks>
2. Kết quả từ thực nghiệm
3. Kết quả từ việc thực hiện mô hình tương đương tại Viettel

Chỉ áp dụng phương pháp này khi có bài toán tham chiếu và bài toán cần định cỡ sử dụng mô hình dựa trên mạng neuron và chung một loại mô hình (ví dụ Llama, Deepseek), chỉ khác nhau về các điều kiện:

- Kích Thước mô hình (Model Size): Số lượng tham số (Parameters)
- Độ chính xác (Precision): FP32 (32-bit), FP16/BF16 (16-bit), INT8 (8-bit)
- KPI về Thông lượng (Throughput) và Độ trễ (Latency)
- Dữ liệu đầu vào (độ phân giải ảnh, độ dài chuỗi, ...)

Giả sử Baseline bài toán A đã biết: mô hình A (có số lượng tham số là $Params_A$), Cần dùng số lượng N_A GPU loại g để đạt Thông lượng đáp ứng Q_A , Độ trễ trung bình L_A .

Yêu Cầu (Bài toán B): mô hình B (có số lượng tham số là $Params_B$), Thông lượng đáp ứng Q_B , Độ trễ trung bình L_B . Nếu cùng sử dụng loại GPU g giống như bài toán A thì cần số lượng là bao nhiêu?

Số GPU loại g cần thiết để giải bài toán B

$$N_B = N_A * \frac{PARAM_B}{PARAM_A} * \frac{Q_B}{Q_A}$$

Công thức này giả định rằng các điều kiện còn lại không thay đổi (precision, dữ liệu đầu vào, ...). Trường hợp các điều kiện này thay đổi thì tính thêm tỉ lệ dựa trên kích thước của các điều kiện này

Lưu ý:

- *Càng nhiều điều kiện khác biệt thì tính chính xác tham chiếu càng giảm.*
- *Trong trường hợp mô hình không dựa trên mạng neuron, cần nghiên cứu tài liệu chuyên ngành hoặc tự giải trình chi tiết cách thức so sánh tương quan phù hợp.*

4. Quy đổi loại GPU

Giả sử đã biết đối với bài toán cần giải quyết cần N GPU loại g. Nếu đổi sang GPU loại g_1 , cần tìm số lượng N_1 GPU loại g_1 để giải quyết bài toán ban đầu.

Các bài toán AI không chỉ đơn thuần là tính toán. Hiệu năng của chúng phụ thuộc vào sự cân bằng giữa khả năng tính toán và tốc độ truy xuất dữ liệu. Hầu hết các bài toán thuộc một trong ba loại dưới đây. Công thức quy đổi GPU sẽ tùy vào từng loại bài toán:

- **Compute-Bound (Giới hạn bởi tính toán):** Tốc độ xử lý bị giới hạn bởi sức mạnh tính toán (TFLOPS) của GPU. GPU luôn "bận rộn" tính toán và không phải chờ đợi dữ liệu.

- **Memory-Bound (Giới hạn bởi bộ nhớ):** Tốc độ xử lý bị giới hạn bởi tốc độ của bộ nhớ (băng thông VRAM). GPU có khả năng tính toán rất nhanh nhưng lại phải dành phần lớn thời gian chờ dữ liệu được nạp từ VRAM.

- **Interconnect-bound (NVLink/NVSwitch/PCIe/IB):** Thông lượng/độ trễ toàn hệ thống bị chi phối bởi băng thông và độ trễ liên kết giữa các GPU (trong server và/hoặc giữa các server). Phần lớn thời gian mỗi bước chạy dành cho truyền thông và đồng bộ (collective comms: all-reduce, all-gather, reduce-scatter, all-to-all) hơn là thực thi tính toán trên các Compute Units (CU) của GPU — nói cách khác, thời gian giao tiếp > thời gian tính toán. Ví dụ thư viện: NCCL/RCCL/oneCCL/MPI/Gloo.

Bước 1: Kiểm tra điều kiện tiên quyết (Threshold Check)

- Kiểm tra: VRAM của 1 GPU g_1 lớn hơn VRAM tối thiểu mà tác vụ yêu cầu thì chuyển sang bước 2.

- Nếu VRAM của 1 GPU g_1 không đủ để suy luận thì phải dùng Model Parallelism. Số GPU g_1 cần thiết để có được tổng VRAM tương đương hệ thống cũ là:

$$N_{1vram} = N * \frac{VRAM_g}{VRAM_{g_1}}$$

Trong đó $VRAM_g$ và $VRAM_{g_1}$ là bộ nhớ của GPU loại g , g_1

Bước 2: Ước tính dựa trên Bottleneck (Bottleneck Estimation)

a) Nếu là Compute-Bound: Số GPU loại g_1 cần thiết để đáp ứng khả năng tính toán

$$N_{1compute} = N * \frac{FLOPS_{gprecision}}{FLOPS_{g_1precision}} * \frac{S_{gsparcity}}{S_{g_1sparcity}}$$

Trong đó

- $FLOPS_{gprecision}$ và $FLOPS_{g_1precision}$ là năng lực tính toán của GPU loại g , g_1 tại định dạng precision theo yêu cầu của bài toán (vd: FP32, BF16, FP8).
- $S_{gsparcity} = 2$ nếu Mô hình/Bài toán tương thích với Sparsity và GPU g có phần cứng hỗ trợ tăng tốc Sparsity, $S_{g_1sparcity} = 2$ nếu Mô hình/Bài toán tương thích với Sparsity và GPU g_1 có phần cứng hỗ trợ tăng tốc Sparsity

Số GPU N_1 cần dùng:

$$N_1 = \text{Max}(N_{1vram}, N_{1compute})$$

b) Nếu là Memory-Bound (ví dụ: Suy luận LLM): Số GPU loại g_1 cần thiết để đáp ứng băng thông:

$$N_{1bw} = N * \frac{BW_g}{BW_{g_1}}$$

Trong đó BW_g và BW_{g_1} là băng thông bộ nhớ của GPU g và g_1 (đơn vị GB/s, thông số này luôn có trong datasheet của GPU).

Số GPU N_1 cần dùng

$$N_1 = \text{Max}(N_{1vram}, N_{1bw})$$

c) Nếu là Interconnect-bound: Số GPU loại g_1 cần thiết để đáp ứng giao tiếp:

$$N_{1link} = N * \frac{BWLink_g}{BWLink_{g_1}}$$

Trong đó $BWLink_g$ và $BWLink_{g_1}$ là băng thông liên kết hữu dụng của GPU g và g_1 (NVLink/NVSwitch trong node; PCIe/IB giữa node).

Số GPU N_1 cần dùng:

$$N_1 = \text{Max}(N_{1vram}, N_{1link})$$

Bước 3: Kiểm tra ràng buộc điện/nhiệt (TDP) và hạ tầng

- Tổng TDP của hệ thống mới: Tổng TDP = $N_1 * TDP_{g1}$
- Làm mát: đảm bảo lưu lượng gió/ liquid cooling tương thích.
- NIC/IB: nếu multi-node, đối chiếu băng thông mạng (200–400–800 Gb/s) với profile all-reduce; nếu thiếu, xem như một “interconnect bottleneck” ở bước 2c.

5. Tổng hợp kết quả

Dựa vào kết quả bước 3 thực hiện tổng hợp các thành phần có thể sử dụng chung tài nguyên phần cứng. Dựa trên các công nghệ, hiện tại, khả năng sẵn sàng, có sẵn của sản phẩm phần cứng tương ứng để xuất hệ thống cho phù hợp.

Lưu ý:

- Trường hợp số lượng GPU không phải số nguyên, cần làm tròn lên. Trường hợp có sử dụng MIG, cần xác định chi tiết thêm số lượng instance có thể chia/hỗ trợ mỗi GPU.
- Sau khi hoàn tất định cỡ, cần bổ sung thêm 10% công suất nhằm bù trừ sai số tính toán.
- Đối với môi trường triển khai suy luận trên hệ thống Production, số lượng GPU hoặc máy chủ GPU phải được thiết kế theo nguyên tắc dự phòng tối thiểu N+1 để đảm bảo tính sẵn sàng và ổn định của dịch vụ.

Lập bảng kết quả sizing chi tiết: (Số liệu mẫu)

Stt	Tài nguyên	Mô tả	Số lượng	Đơn vị	Loại GPU
<i>I. Tài nguyên huấn luyện, phân hệ A</i>					
1	GPU	Loại card, cấu hình	2	GPU	H100
<i>II. Tài nguyên suy luận, phân hệ B</i>					

Dựa trên kết quả tính toán chi tiết, kết hợp với các tài liệu của thiết bị, tham khảo/ khuyến nghị từ hãng cung cấp thiết bị, cung cấp hạ tầng GPU để xác định tiếp số lượng GPU và server (hoặc server ảo) thực tế cần thiết

Kết quả tổng hợp tài nguyên định cỡ: (Số liệu mẫu)

Stt	Tài nguyên	Mô tả cấu hình	Số lượng	Đơn vị	Ghi chú
1	GPU server phục vụ huấn luyện	DGX, 8x Card H100 80GB	01	Server	
2	GPU server phục vụ suy luận	Server với 8xL40s	02	Server	

Yêu cầu hạ tầng vật lý tổng thể:

Stt	Hạng mục	Mô tả chi tiết	Giá trị ước tính	Đơn vị	Ghi chú
1	Công suất điện	- 1 server DGX H100: ~10.2 kW - 2 server L40S (mỗi server ~4.5 kW): 9.0 kW	~19.2	kW	Cần ít nhất 2 PDU 32A/3-phase cho cụm này.
2	Tản nhiệt	19.2 kW * 3412 BTU/kW	~65,510	BTU/giờ	Yêu cầu hệ thống làm mát công suất cao.
3	Không gian Rack	- 1 server DGX H100: 8U - 2 server L40S (mỗi server 4U): 8U	16	U	Cần 1 tủ rack 42U tiêu chuẩn.
4	Dự phòng UPS	> 125% của 19.2 kW	> 24	kVA	Đảm bảo an toàn khi mất điện lưới.